Application of distributed neural networks for anomaly detection in large-scale industrial IoT data.

Adriana Silva de Souza
(CCT Colege)
Dublin, Ireland
Adrianasilvadesouza0@gmail.com

***Abstract*— This article presents two sets of experiments conducted using the Edge-IIoTset dataset to investigate, in an integrated manner, Big Data storage and processing, as well as the detection of cyberattacks within Industrial Internet of Things (IIoT) environments. The study includes comparisons between CSV and Apache Parquet, and between pandas and PySpark, alongside experiments involving Multilayer Perceptron (MLP) neural networks, Dropout regularization, and the optimization of classification thresholds. The analysis considers computational performance, scalability, predictive metrics, and the trade-off between false positives and false negatives. The results should be interpreted in light of the execution conditions within Google Colab and the methodological limitations documented in the notebooks.**

Palavras-chave: Industrial IoT; Edge-IIoTset; Big Data; Apache Spark; Parquet; MLP; detecção de ataques.

I. INTRODUCTION (*HEADING 1*)

The security of Industrial Internet of Things (IIoT) environments entails both the capacity to store and process large volumes of data and the identification of malicious behavior within network traffic. The two notebooks that form the basis of this article investigate these dimensions using the Edge-IIoTset dataset. The first examines storage formats, execution mechanisms, and computational performance across varying volumes of records. The second addresses the binary classification of normal versus malicious traffic using Multi-Layer Perceptron (MLP) neural networks, comparing different architectures and decision thresholds. This article presents the experiments as a cohesive sequence, incorporating selected excerpts from the original notebooks directly into the methodology and results sections.

II. DATASET AND PREPARATION

The dataset used in the work presented here is called Edge-IIoTset, developed by Ferrag et al. (2022). Its primary objective is to support research related to cybersecurity within Internet of Things (IoT) and Industrial Internet of Things (IIoT) environments.

Edge-IIoTset was constructed using an experimental infrastructure that integrates IoT/IIoT devices, communication protocols, and various cyberattack scenarios. The collected records represent network traffic characteristics, enabling the investigation of patterns associated with both legitimate and malicious activities (Ferrag, 2022). This dataset was selected because it aligns well with the specific objectives of this research, providing data that can be utilized both for Big Data processing experiments and for the development of anomaly detection models based on neural networks.

Another positive aspect of this dataset is that it is publicly available—specifically on Kaggle—which facilitates its use in academic research. The original file contains a total of 2,219,201 records and 63 columns, as identified in the tests described below.

Based on these figures, we were able to begin investigating strategies for data storage, management, and processing at various scales.

A key strength of this work is the ability to conduct diverse experiments using varying numbers of records, allowing for an evaluation of how different technologies perform as the volume of data increases. In this work, we address three main concepts:

- Storage -> We compare how CSV and Parquet formats utilize storage space, as well as the time required to read and write data.
- Processing -> We compare the execution times of pandas and PySpark as the number of records increases.

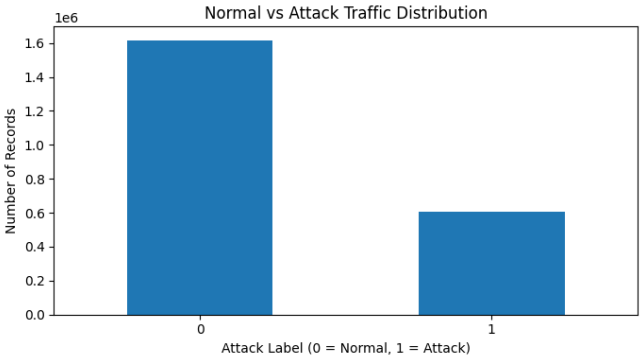


Fig. 1. Distribuição das classes no Edge-IIoTset. Fonte: gráficos e diagramas originais dos notebooks.

For this project, we selected a dataset containing approximately 2.2 million records. From this, we drew samples of varying sizes to conduct our experiments, as follows:

- 200,000 records: selected to evaluate performance with a smaller volume of data.

- 500,000 records: selected to observe behavior with an intermediate volume of data.

- 1,000,000 records: selected to investigate the impact of a larger volume on total processing time.

The selected dataset contains 63 columns, including attributes related to network traffic, protocol information, and variables used to identify malicious activity (Ferrag, 2022).

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

During the exploratory testing phase, it was observed that the dataset contains two classification variables that proved to be highly promising, as they provide information regarding the presence and nature of malicious activity within the network traffic:

Attack_label: identifies whether a record represents normal traffic (labeled 0) or malicious traffic (labeled 1).

Other studies have explored these two variables—for instance, the research by Ferrag et al. (2022), who created the Edge-IIoTset and developed these labels to support the creation and evaluation of machine learning-based intrusion detection systems.

Subsequent studies have also utilized this classification structure. An interesting example can be found in the paper "A deep learning approach for multiclass attack classification in IoT and IIoT networks using convolutional neural networks" (2025), which investigated various classification levels, including binary detection and the identification of specific attack categories.

Based on this information, we have identified these two variables as being highly significant for our current work; they enable the development and analysis of binary anomaly detection models, while also facilitating the identification of specific attack types and the creation of multi-class classification models.

By distinguishing between these two variables, we aim to understand both the presence of malicious activity and its diversity, thereby establishing a solid foundation for data preparation and subsequent anomaly detection experiments.

An important note: The Attack_label variable was used to perform stratified sampling, ensuring that the class distribution was preserved throughout the experiments.

Following the acquisition and verification of the DNN-EdgeIIoT-dataset.csv file, an initial exploratory analysis was conducted using the pandas library to better understand the structure and characteristics of the available data.

For this stage, the first 100,000 records were loaded, corresponding to an initial sample of the original file, which contains approximately 2.2 million observations.

The dimensions (100,000, 63) indicate that the DataFrame consists of 100,000 rows and 63 columns. It is important to note that the memory consumption of 102.52 MB reflects how the data is represented within the Python environment, rather than the size of the original CSV file.

The preview of the first five records, shown below, reveals attributes associated with various communication protocols—such as ARP, ICMP, and MQTT—along with information including source and destination IP addresses and timestamps.

ARP, ICMP, and MQTT are communication protocols used in computer networks and IoT (Internet of Things) devices.

Each serves a distinct function in facilitating communication between devices (MINOLI, 2019).

In our project using the Edge-IIoTset dataset, these protocols are significant because they represent different characteristics of network traffic that can be analyzed to distinguish between normal behavior and potential cyberattacks.

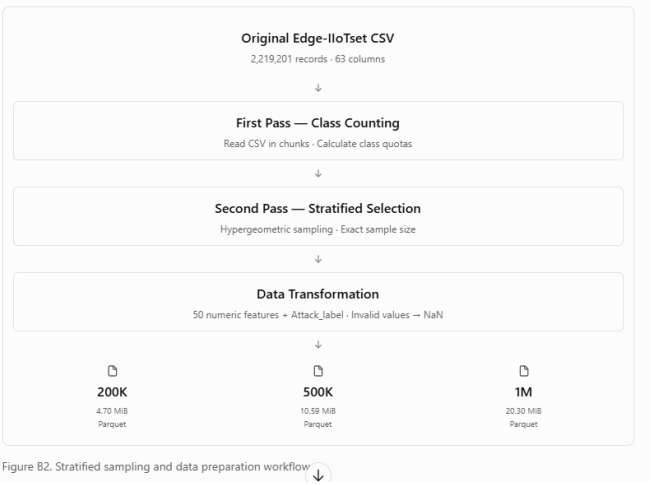


Fig. 2. Stratified sampling procedure and data preparation. Source: original charts and diagrams from the notebooks.

An important point noted is that the observed memory consumption highlights the significance of considering available computational resources when running experiments.

In summary, the initial exploration confirmed the tabular structure of the Edge-IIoTset and allowed us to identify the diversity of attributes present in the dataset. With this information, we were able to establish a solid foundation for the subsequent stages, which include:

Analysis of data types:

- Class distribution
- Feature selection
- Stratified sampling.

Following an initial exploration of the Edge-IIoTset, we analyzed the structure of the DataFrame using the `info()` function from the pandas library, with the objective of identifying:

The results show a predominance of numerical attributes, which account for approximately 81% of the columns, while the remaining attributes consist of textual information, such as IP addresses, protocol identifiers, and attack categories.

The analysis also identified an absence of null values within the 100,000 selected entries. However, this observation does not guarantee the total absence of invalid, infinite, or missing values in other parts of the dataset.

III. ARMAZENAMENTO E PROCESSAMENTO DE BIG DATA

Following the initial exploration and identification of the numerical attributes in the Edge-IIoTset, a stratified sampling strategy was implemented to prepare representative datasets for storage and processing experiments.

The need for this approach became apparent during Experiment B1, when it was observed that the first 100,000 records in the original file belonged exclusively to the "normal" traffic class. However, a full analysis of the dataset revealed a distribution of approximately 72.8% normal records and 27.2% malicious records.

This discrepancy highlighted that selecting the initial records sequentially could result in non-representative samples. Consequently, the `Attack_label` variable was used as a stratification criterion, ensuring that the original proportion between the two classes was approximately maintained.

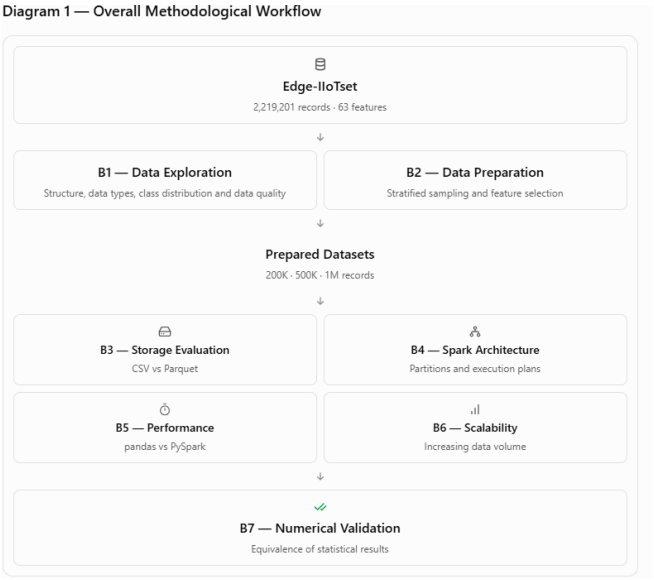


Fig. 3. Methodological workflow for the Big Data experiments. Source: original charts and diagrams from the notebooks.

The procedure was implemented using the `stratified_csv_sample()` function, developed in Python to perform sampling without loading the entire CSV file into memory.

The function employs a chunk-based processing strategy, handling blocks of up to 100,000 records. Initially, the file is traversed to count the number of records belonging to each class. Subsequently, the proportional quotas required to produce a sample of the requested size are calculated.

In a second pass through the file, records are randomly selected within each class, using a hypergeometric distribution to control the number of observations chosen from each block. This strategy allows for the creation of a sample of an exact size, while adhering to the established class quotas.

Following the sampling process, the attributes selected in Experiment B1 were converted into numerical formats.

Values incompatible with this representation—as well as any infinite values—were transformed into missing values (NaN).

The `Attack_label` variable was kept separate as classification information, while the 50 numerical attributes were preserved for subsequent analytical operations.

The procedure initially produced a sample of 1,000,000 records, which served as the basis for generating three experimental datasets.:

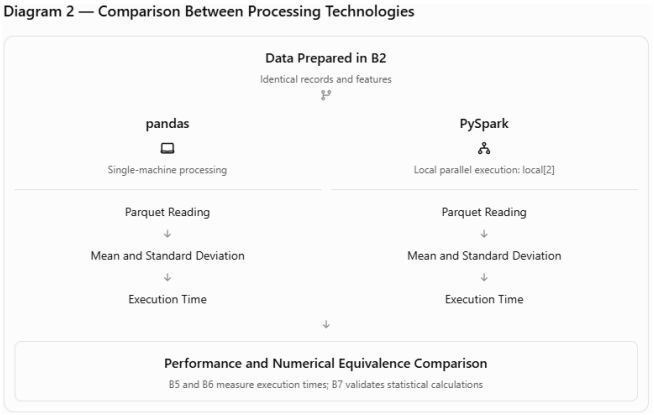


Fig. 4. Comparação das tecnologias de processamento. Fonte: gráficos e diagramas originais dos notebooks.

Using varying volumes of records makes it possible to investigate how an increase in the amount of data influences storage requirements and computational performance.

Furthermore, preparing datasets with an identical attribute structure allows for consistent comparisons between pandas and PySpark, minimizing methodological differences that could otherwise compromise the interpretation of the results.

Following the preparation of the datasets in Experiment B2, a comparative evaluation of the CSV (Comma-Separated Values) and Apache Parquet formats was conducted to examine the impact of the storage structure on disk space usage and the performance of data input/output operations.

CSV is a text-based format widely used for storing and exchanging tabular data. Its primary advantages lie in its simplicity, portability, and compatibility with a variety of tools. However, its textual representation can increase the space required to store numerical values and may necessitate additional interpretation and conversion steps during data reading.

Apache Parquet, by contrast, employs columnar storage, organizing values according to the table's attributes. This structure facilitates encoding and compression mechanisms, while also enabling analytical tools to retrieve only the specific columns required for a given operation.

Due to these characteristics, Parquet is frequently used in architectures designed for the analytical processing of large volumes of data (Apache Parquet, n.d.).

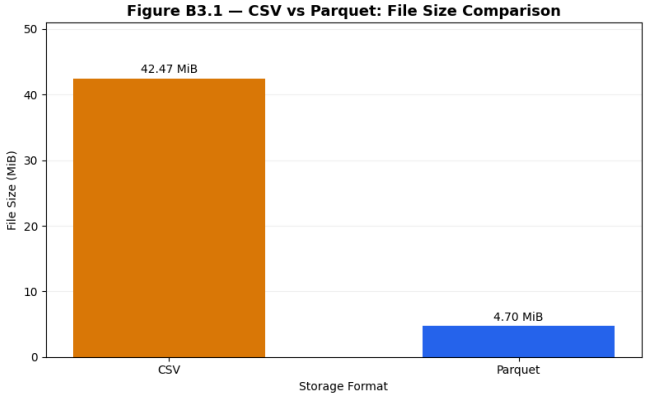


Fig. 5. Comparison of CSV and Parquet file sizes. Source: original charts and diagrams from the notebooks

To compare the two formats, a dataset consisting of 200,000 records and 51 columns—prepared for Experiment B2 and containing 50 numerical attributes plus the classification variable `Attack_label`—was used.

1. Storage space: measurement of the file size for both CSV and Parquet formats, expressed in MiB.

2. Write time: measurement of the time required to write the same dataset in each format.

3. Read time: measurement of the time required to load the files, based on five timed runs following an initial warm-up read.

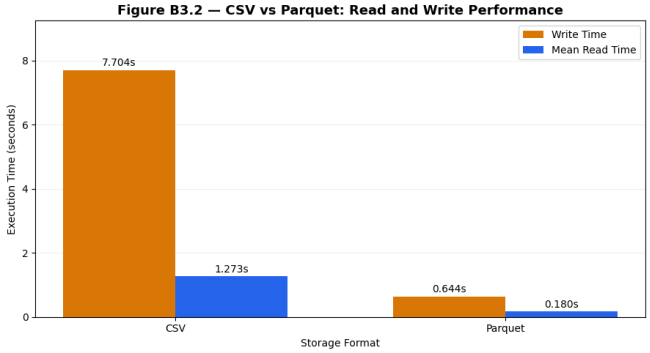


Fig. 6. Read and write times for CSV and Parquet formats. Source: original charts and diagrams from the notebooks.

Measurements were performed using the `time.perf_counter()` function, which is suitable for measuring execution intervals using a high-resolution clock.

For reading times, the mean and standard deviation across five runs were calculated, allowing for an assessment of both average performance and observed variability.

The results were organized into a DataFrame and exported to a CSV file, facilitating their subsequent use in a comparative analysis.

It is important to note that this experiment evaluates read and write operations performed using pandas within the Google Colab environment.

Therefore, the results do not represent a direct comparison between distributed storage mechanisms, but rather an experimental evaluation of file formats under local conditions.

The execution of Experiment B3 enabled a comparison of storage space as well as read and write times for the CSV and Apache Parquet formats, using the same dataset consisting of 200,000 records and 51 columns.

The results demonstrate that the Apache Parquet format required less storage space and yielded faster read and write times under the conditions evaluated.

The CSV file took up approximately 42.47 MiB, while the Parquet file occupied 4.70 MiB, representing a reduction of approximately 88.94% in the space used.

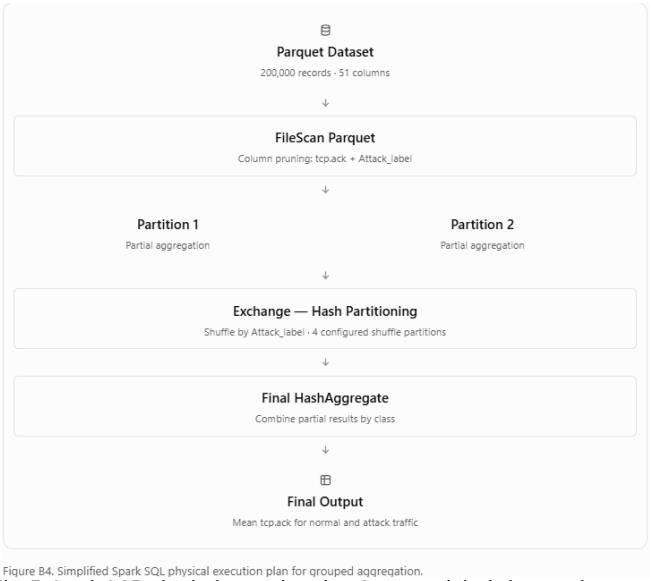


Fig. 7. Spark SQL physical execution plan. Source: original charts and diagrams from the notebooks

This difference can be explained by the textual representation of values in CSV files and the encoding and compression mechanisms employed by the columnar Parquet format.

In this experiment, writing data to Parquet was approximately 11 times faster than writing to CSV.

This result demonstrates an advantage for Parquet under the tested conditions. However, since only one write measurement was taken for each format, it is not possible to statistically evaluate the variability of these durations.

The standard deviation was also lower for Parquet, indicating less absolute variation in read times across the five observed runs.

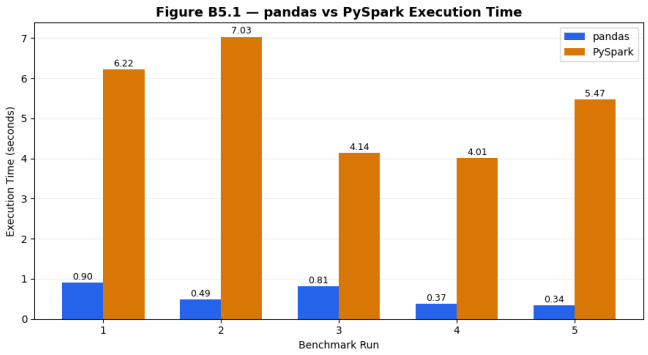


Fig. 8. Comparison of execution times for pandas and PySpark. Source: original charts and diagrams from the notebooks.

In general, the results indicate that Apache Parquet was more efficient than CSV for storing and retrieving the numerical dataset used in this experiment.

Reducing file size can help lower storage and data transfer requirements, while faster read times can benefit repetitive analytical operations.

These characteristics justify the use of Parquet in subsequent experiments involving pandas and PySpark, as it allows both technologies to work with a consistent data representation.

It is important to note that these results do not imply that Parquet will necessarily be superior in every scenario. Performance depends on factors such as data types, compression, hardware, the specific library used, and the characteristics of the workload.

Therefore, Experiment B3 provides evidence that—given the numerical dataset selected for the experiment and the specific computing environment evaluated—Apache Parquet offered advantages in terms of storage and performance compared to CSV.

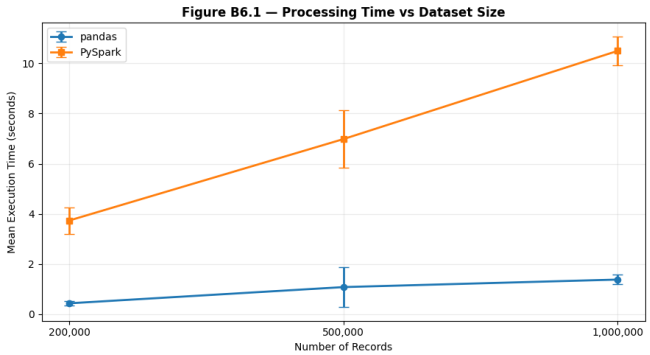


Fig. 9. Scalability of processing time by volume. Source: original charts and diagrams from the notebooks.

Following the evaluation of storage formats in Experiment B3, an investigation into Apache Spark's processing mechanisms was conducted to understand how the framework organizes and executes analytical operations on structured data.

Apache Spark is a data processing engine that supports parallel and distributed execution. Its architecture allows for the division of operations into tasks, the processing of data partitions, and the combination of results through aggregation mechanisms and inter-process communication (Zaharia et al., 2016).

A key feature of Spark is the use of Spark SQL, which enables operations on structured data via the DataFrame API. Spark SQL incorporates query optimization mechanisms and execution plan selection, which help reduce unnecessary operations and organize processing efficiently (Armbrust et al., 2015).

In this experiment, a dataset containing 200,000 records and 51 columns—previously prepared in the Apache Parquet format—was used.

The Spark environment was configured in Google Colab using the local parallel execution mode (`local[2]`), which provides two execution threads for processing tasks.

The experimental procedure involved reading the Parquet file, identifying the number of partitions, and performing a statistical aggregation operation.

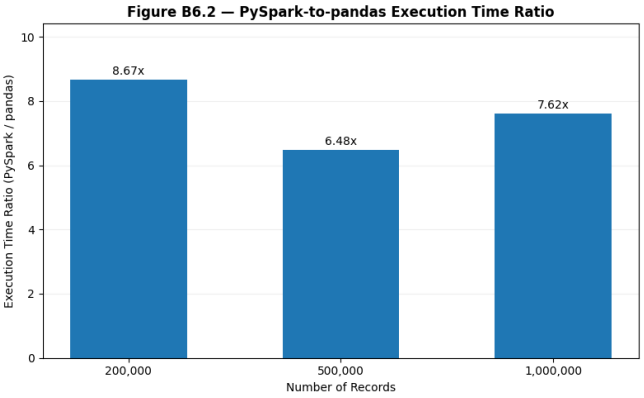


Fig. 10. Ratio between PySpark and pandas execution times. Source: original charts and diagrams from the notebooks

. For this analysis, the records were grouped according to the `Attack_label` variable, with the mean of the numerical attribute `tcp.ack` calculated for each class.

In the optimized logical plan, a `Project` operation was identified, which is responsible for selecting the attributes required for processing.

IV. NEURAL NETWORKS AND ATTACK DETECTION

In this phase, our objective is to implement a stratified random sampling procedure to select a representative subset of samples from the Edge-IIoTset dataset. We aim to preserve the original proportion between normal and malicious traffic, while also avoiding the need to load the entire CSV file into memory at once.

As previously mentioned, the Edge-IIoTset was developed by Ferrag et al. (2022) to support cybersecurity research in IoT and Industrial IoT environments. The dataset includes various categories of attacks and provides an `Attack_label` variable, which allows for the distinction between normal (0) and malicious (1) traffic records.

1. Class representativeness: Selecting only the initial records from a file can introduce bias if the data is ordered by category or traffic type.

2. Computational resource management: Selecting a subset reduces the memory required for training experiments.

3. Experimental consistency: Using the same sample across experiments E1, E2, and E3 enables the comparison of models under equivalent conditions.

The importance of stratification in evaluating classifiers is discussed by Kohavi (1995), who investigated validation strategies and performance estimation. Although the author focused primarily on cross-validation methods, his research provides a conceptual basis for preserving class composition.

To implement this strategy, we developed a function called `stratified_csv_sample()`, which incorporates the following:

- Incremental reading using pandas.

- Proportional quota calculation.

- Random selection without replacement, based on a hypergeometric distribution.

- A "seed" parameter, which controls the sequence of generated numbers and ensures the reproducibility of the selection process.

Defining a fixed seed is crucial because subsequent experiments depend on the specific composition of the selected sample. It is important to note that, although the selection is random, it follows a sequence determined by a seed. This means that while we can test different architectures, they will all be trained using the same samples, providing a more reliable basis for comparison.

- If each architecture were trained using different samples, it would be difficult to determine whether performance differences were due to the architecture itself or to the specific characteristics of the selected records.

This decision relates to the need for consistency in the experimental evaluation of classifiers, a topic discussed by Kohavi (1995). However, Kohavi does not specify the value 42 as a standard; the choice of this particular seed is an implementation decision specific to this project.

The purpose of this code block is to determine the number of records belonging to each class in the original dataset:

- `usecols=[label]` restricts the data read to the classification variable, avoiding the need to load attributes that are unnecessary for this step.

- `value_counts()` tallies the records for each class within the current data block, while `counts.add()` accumulates these totals across successive reads.

- `fill_value=0` ensures that counts can be combined even when a specific class is absent from one of the blocks.

Incremental reading was chosen to reduce memory consumption when analyzing a large-scale file.

This approach makes it possible to determine the overall class distribution without having to store every observation from the CSV file in memory simultaneously.

Once the total number of records for each class (Normal and Attack) has been tallied, it is possible to identify the overall distribution of classes within the dataset.

- This information is necessary to calculate the proportion of each class and to determine how many records should be selected during the stratified sampling process.

Initially, the algorithm calculates the expected number of records per class by multiplying its original frequency by the total sampling fraction.

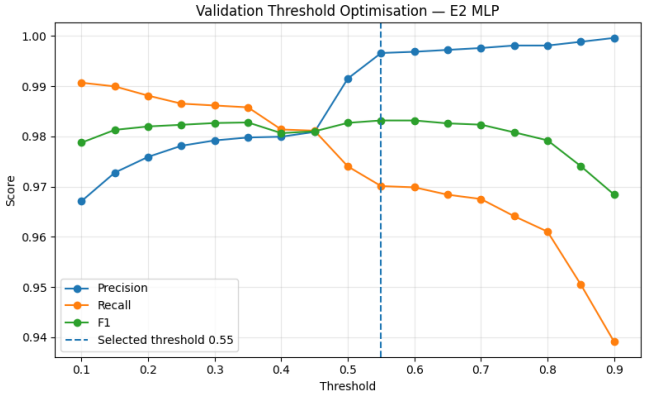


Fig. 11. Optimization of the decision threshold on the validation set. Source: original charts and diagrams from the notebooks.

- (E_c) represents the expected number of records for the class.
- (N_c) represents the original number of records for the class.
- (n) represents the desired sample size.
- (N) represents the total number of records in the dataset.

Since the number of selected records must be an integer, `np.floor()` is used to initially round the quotas down.

For this reason, the algorithm calculates the difference using the `remainder` variable and distributes the remaining records among the classes that exhibited the largest fractional parts.

The use of proportional quotas was adopted to preserve, as closely as possible, the original distribution between normal and malicious traffic.

These structures are updated during the second pass through the file, allowing the algorithm to take into account the selections made previously.

- `pop`: the number of records from that class remaining in the as-yet-unprocessed set.

- `quota`: the number of records from that class that still need to be selected.

- `available`: the number of records from that class available in the current block.

This distribution represents the number of elements of a given category obtained in a random sample drawn from a finite population without replacement.

- \(N\): the number of records from the class remaining.

- \(K\): the number of records still required to fulfill the quota.

- \(n\): the number of records from the class available in the block.

- \(k\): the quantity drawn for selection.

This mechanism allows for the distribution of random selection across blocks, rather than establishing a fixed number of records per block.

The broader problem of random selection without replacement in large datasets is also investigated by Vitter (1985).

It is important to note that the implementation in this notebook does not use the reservoir sampling algorithm proposed by Vitter. Instead, the code combines a two-pass strategy with hypergeometric draws and random selection within each block.

After determining the number of records to be selected, the algorithm uses `group.sample()` to randomly choose the corresponding observations.

The use of `random_state`—derived from the previously initialized random number generator—allows for control over the randomness of the procedure.

The `remaining_population` variable is reduced by the number of records processed in the block, while `remaining_quota` is reduced by the number of records actually selected. However, this does not, in itself, prove that the class distribution has been preserved or that the selection is statistically representative.

This value was chosen as a practical setting to manage memory consumption; it does not represent an experimentally determined optimal size.

Implementing stratified sampling is an essential step in preparing experiments involving neural networks.

By preserving, to a close approximation, the original distribution between normal and malicious traffic, this procedure reduces the risk of distortions that might arise from a sequential selection of records.

Using a sample of controlled size also allows the experiments to be adapted to the computational limitations of Google Colab.

Furthermore, using the same sample across experiments E1, E2, and E3 facilitates a more consistent comparison between different neural architectures and classification strategies.

The objectives of this section are to:

- define the classification problem to be explored;

- justify the choice of neural architecture;

- establish an appropriate dataset for subsequent experiments on malicious traffic detection.

The Edge-IIoTset dataset, originally developed by Ferrag et al. (2022), was created to investigate the ability of artificial neural networks to distinguish between normal and malicious behavior in Industrial Internet of Things (IIoT) environments.

Our choice is based on the original structure of the Edge-IIoTset, which includes an `Attack_label` variable specifically designed for this purpose (Ferrag et al., 2022).

Accordingly, we aim to develop a model capable of learning the relationships between the numerical characteristics of network traffic and the presence of malicious activity.

Taking all the preceding discussion into account, we have selected the Multilayer Perceptron (MLP)—a type of feedforward artificial neural network—as our initial architecture. An MLP consists of an input layer, one or more hidden layers, and an output layer. During training, the parameters are adjusted to learn relationships between the input attributes and the expected classification values. (Scikit-learn developers, 2026)

According to Goodfellow, Bengio, and Courville (2016), feedforward neural networks can learn non-linear functions through the composition of multiple transformations, enabling the representation of complex relationships between variables.

In the work we are proposing, the choice of an MLP (Multilayer Perceptron) was motivated by the use of fixed-length, tabular numerical attributes—extracted from network traffic—found in the dataset we are using as a basis.

This representation allows each record to be used as a feature vector, without requiring the prior construction of temporal sequences or spatial structures.

Ferrag et al. (2022) also investigated deep learning models using the Edge-IIoTset, providing a reference directly related to the application of neural networks for intrusion detection.

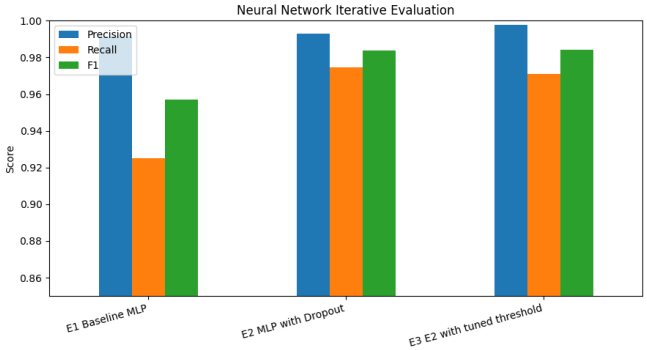


Fig. 12. Comparison of precision, recall, and F1 in experiments E1, E2, and E3. Source: original charts and diagrams from the notebooks.

Although we selected the MLP for experiments E1 and E2, other architectures could easily be considered. These include

Convolutional Neural Networks (CNNs): These utilize convolutional operations to exploit local relationships (Google Cloud, 2026). Applying them to the tabular attributes of the Edge-IIoTset would require justifying an organization of the attributes where proximity between positions holds meaningful significance—a condition not met by the attributes in the Edge-IIoTset, which are tabular variables that do not necessarily possess a meaningful proximity relationship.

It is important to note that while using CNNs on tabular data is possible, it was not experimentally investigated in this notebook.

Recurrent Neural Networks (RNNs), LSTMs, and GRUs: These architectures are well-suited for modeling sequential dependencies (Coursera, 2026). They could be employed if the records were organized into valid temporal sequences, thereby preserving information regarding the evolution of traffic. However, the representation used in this study considers individual records without explicitly constructing such sequences.

Transformers: These can model relationships between elements in sequences through attention mechanisms (IBM, 2025). Utilizing them would require defining an appropriate data representation and evaluating whether the benefits justify the added complexity.

Autoencoders: These can learn compressed representations of data and be used in anomaly detection strategies based on reconstruction errors (DataCamp, 2025).

They represent a relevant alternative for scenarios with limited label availability, though they were not implemented in the experiments presented here.

It is important to emphasize that selecting the MLP for these experiments does not imply that it is superior to the other neural networks mentioned above. Rather, it is a methodological choice based on the dataset used, the format of the attributes, and the objectives of the investigation. - `nrows=10_000` limits the number of records loaded, allowing for an initial inspection of the dataset's structure without reading the entire file.

- It is important to note that the value of 10,000 records is a practical choice—as we are only conducting a preliminary inspection—and does not represent a statistically optimized sample size.

- The `assert` statement verifies whether the variables `Attack_label` and `Attack_type` are present in the dataset.

- This validation was included because these variables serve specific functions within the structure of the Edge-IIoTset, as described by Ferrag et al. (2022).

- We use `Attack_label` as a reference for binary classification, while `Attack_type` represents the specific category of the traffic or attack.

Performing this check helps prevent subsequent steps from being executed on a file that lacks the expected labels.

* However, the mere presence of these columns does not, by itself, guarantee that their values are correct. For this reason, the target variable will also undergo a numerical conversion in the next step.

The code above was developed to identify numerical attributes that can be used as inputs for neural models.

- The `difference()` method excludes the `Attack_label` and `Attack_type` variables from the list of predictor attributes.

- This exclusion is essential because both variables represent information related to the classification itself.

- Including `Attack_label` as an input would allow the model to directly access the answer it is supposed to learn to predict.

* Therefore, both variables must be excluded from the model's inputs.

The distinction between predictor attributes and the target variable relates to the fundamentals of supervised learning as discussed by Goodfellow, Bengio, and Courville (2016).

This result shows that 50 candidate numerical attributes have been selected to represent each network traffic record.

- These attributes form the initial basis for constructing the input vectors for the MLP (Multilayer Perceptron).

Preserving the class composition is also related to the principles for evaluating classifiers as discussed by Kohavi (1995).

We suggest that you use a text box to insert a graphic (which is ideally a 300 dpi TIFF or EPS file, with all fonts embedded) because, in an MSW document, this method is somewhat more stable than directly inserting a picture.

To have non-visible rules on your frame, use the MSWord “Format” pull-down menu, select Text Box > Colors and Lines to choose No Fill and No Line.

REFERENCES

[1] Ferrag, Mohamed Amine, et al. "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning." IEEe Access 10 (2022): 40281-40306.

[2] Seyedkolaei, Ali Abdi, Fatemeh Mahmoudi, and José García. "A deep learning approach for multiclass attack classification in IoT and IIoT networks using convolutional neural networks." Future Internet 17.6 (2025): 230. -< https://www.mdpi.com/1999-5903/17/6/230

[3] McKinney, W. (2010) 'Data Structures for Statistical Computing in Python', Proceedings of the 9th Python in Science Conference, pp. 56–61. https://doi.org/10.25080/Majora-92bf1922-00a

[4] Zaharia, M., Xin, R.S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M.J., Ghodsi, A., Gonzalez, J., Shenker, S. and Stoica, I. (2016) 'Apache Spark: A Unified Engine for Big Data Processing', Communications of the ACM, 59(11), pp. 56–65. https://dl.acm.org/doi/fullHtml/10.1145/2934664

[5] Goldberg, D. (1991) 'What Every Computer Scientist Should Know About Floating-Point Arithmetic', ACM Computing Surveys, 23(1) pp. 5–48. https://doi.org/10.1145/103162.103163

[6] Ferrag, M.A., Friha, O., Hamouda, D., Maglaras, L. and Janicke, H. (2022) 'Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications for Centralized and Federated Learning', IEEE Access, 10, pp. 40281–40306. Available at: https://doi.org/10.1109/ACCESS.2022.3165809

[7] McKinney, W. (2010) 'Data Structures for Statistical Computing in Python', Proceedings of the 9th Python in Science Conference, pp. 56–61. Available at: https://doi.org/10.25080/Majora-92bf1922-00a

[8] Kohavi, R. (1995) 'A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection', Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI), pp. 1137–1145. Available at: Artigo completo em PDF

[9] Vitter, J.S. (1985) 'Random Sampling with a Reservoir', ACM Transactions on Mathematical Software, 11(1), pp. 37–57. Available at: https://doi.org/1sq0.114475/3147.3165

[10] Pandas development team (2026) pandas.read_csv — pandas documentation. Available at: pandas.read_csv documentation

[11] Ferrag, M.A., Friha, O., Hamouda, D., Maglaras, L. and Janicke, H. (2022) 'Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications for Centralized and Federated Learning', IEEE Access, 10, pp. 40281–40306. https://doi.org/10.1109/ACCESS.2022.3165809

[12] Kohavi, R. (1995) 'A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection', Proceedings of the 14th International Joint Conference on Artificial Intelligence, pp. 1137–1145. Available at: Artigo completo em PDF